

搜索和查询&分析器&聚合

索引-Index

类型-Type: 7.x 弱化 8.x完全删除

文档-Document

创建索引: PUT /index?pretty

查询索引: GET _cat/indices?v GET /index/_search

GET /index/_mapping

删除索引: DELETE /index

插入/更新数据:

PUT /index/_doc/id

{ }

删除数据 DELETE /index/type/id

指定字段更新:

POST /index/_doc/id/_update

{ doc:{ } }

数据类型

① 常见类型

1) 数字:

long integer short byte double float half_float scaled_float unsigned_long

2) Keyword:

Keyword类型的值作为一整体存在倒排索引中, 不进行分词(精准匹配)。

Keyword适合存结构化数据, 如性别、手机号、标签等。字段常用来精确查询、过滤、排序、聚合时, 应设为keyword。

如果某个字段你经常用来做range查询, 你还是设置为数值型(integer,long), ES对数字的range有优化。还可以把字段设为multi-field, 这样又有keyword类型又有数值类型, 方便多种方式使用。

放入倒排索引时, 只截取前一段字符串, 长度由ignore_above参数决定, 默认"ignore_above": 256。

3) Dates (时间类型): 包括 date 和 date_nanos

4) alias: 为现有字段定义别名

5) binary(二进制): binary

6) range(区间类型): integer_range、float_range、long_range、double_range、date_range

7) text: 当一个字段是要被全文搜索的, 比如产品描述, 这些字段应该使用text类型。设置text类型后, 字段内容会被分析, 在生成倒排索引前, 字符串会被分析器分成一个个词项。text类型的字段不用于排序和聚合, 故不需要正排索引。

② 对象关系类型:

1) object: 用于单个JSON对象

2) nested: 用于JSON对象数组

③ 结构化类型:

1) geo-point: 纬度/经度积分

2) geo-shape: 用于多边形等复杂形状

3) point: 笛卡尔坐标点

2) shape: 笛卡尔任意几何图形

④ 特殊类型:

IP地址

completion: 提供自动完成建议

histogram: histogram 用于百分位数聚合的预聚合数值。

array: 数组不是专门的数据类型。默认情况下，任何字段都可以包含零个或多个值，但是，数组中的所有值都必须具有相同的数据类型。

dynamic field mapping:

整数->long

浮点数->float

true | false->boolean

日期->date

数组->取决于数组中的第一个有效值

对象->object

字符串-> 如果不是数字和日期类型，那会被映射为text和keyword两个类型

除了上述字段类型之外，其他类型都必须显示映射，也就是必须手工指定，因为其他类型无法自动识别。

explicit field mapping:

PUT /product2

```
{"mappings":{
```

```
"properties":{
```

```
"date":{
```

```
  "type":"text",
```

```
}}}}
```

映射参数

- index: 是否对当前字段创建倒排索引，默认true，如果不创建索引，该字段不会通过索引被搜索到。
- analyzer: 指定分析器 (character filter、tokenizer、Token filters)
- boost: 对当前字段相关度的评分权重，默认1

- `coerce`: 是否允许强制类型转换, 默认`true` `"1"=> 1`
- `copy_to`: 该参数允许将多个字段的值复制到新字段中, 然后可以将其作为单个字段进行查询
- `doc_values`: 为了提升排序和聚合效率 (正排索引), 默认`true`, 如果确定不需要对字段进行排序或聚合, 则可以设置为`false`以节省磁盘空间 (不支持`text`)
- `ignore_malformed`: 忽略类型转换异常
- `fielddata`: 在JVM中创建临时正排索引

元数据

PUT product

```
{"mappings":{
  "_source":{
    "includes":["name","price"],
    "excludes":["desc","tags"]
  }}}
```

GET product/_search

```
{"_source":false,
"query":{"match_all":{}}}
```

GET product/_search

```
{"_source":["user.name","user.sex","a.*","id"],
"query":{"match_all":{}}}
```

GET product/_search

```
{ "_source": {  
  "includes": ["owner.*", "name"],  
  "excludes": ["desc", "price"],  
  "query": { "match_all": {} }  
}
```

精准匹配=不可分词

#带参数:

GET /product/_search?q=name:xiaomi

#分页:

GET /product/_search?from=0&size=2&sort=price:asc

#精准匹配 exact value

GET /product/_search?q=date:2023-06-01

#_all搜索 相当于在所有有索引的字段中检索

GET /product/_search?q=2023-06-01

GET /product/_search?q=01

match全文搜索

//任一满足

GET product/_search

```
"query": { "match": {  
  "name": "xiaomi nfc phone"  
}}
```

//多个term按搜索顺序在一起，越近分数越高。可以限制它们之间的距离。

GET product/_search

```
{"query": {  
  "match_phrase": {  
    "name": "xiaomi nfc"  
  }}  
}}
```

GET _analyze

```
{"analyzer": "standard",  
"text": "xiaomi nfc phone"}
```

GET /product/_search

```
{"query": {  
  "match_phrase": {  
    "name": {  
      "query": "xiaomi phone1", "slop": 1}}}  
}}
```

//笛卡尔积

GET product/_search

```
{"query": {  
  "multi_match": {  
    "query": "phone huangmenji",  
    "fields": ["name", "desc"]  
  }}  
}}
```

term和match_phrase区别：

match_phrase 会将检索关键词分词,match_phrase的分词结果必须在被检索字段的分词中都包含，而且顺序必须相同，而且默认必须都是连续的

Term搜索不会将搜索词分词，可以和keyword配合使用

term和keyword区别：

term是对于搜索词不分词， keyword是字段类型,是对于source的字段值不分词

//类似SQL中的in语法

GET product/_search

```
{"query": {  
  "terms": {  
    "tags": ["88vip", "gongjiaoka" ]  
  }  
}}
```

GET product/_search

```
{"query":{"range":{"price":{"  
"gte":399,"lte":4999  
}}}}
```

GET product/_search

```
{"query":{"range":{"  
"date":{"gte":"now-1d/d","lte":"now/d"  
}}}}
```

GET product/_search

```
{"query":{"range":{"date":{"  
"time_zone":"+08:00", //增加ES里的字段时间  
"gte":"2023-04-15T08:00:00",//限制查询时间当日为8点 默认是0点
```

```
"lte":"2023-04-16"
```

```
}}}}
```

filter

和query功能一样，只是不关注score，性能比query好很多，另一个用途是对数据的预处理，如提前过滤一些无关数据。

GET product/_search

```
{"query": {  
  "constant_score": {  
    "filter": {  
      "term": {  
        "name": "phone"  
      },  
      "boost": 1.2  
    }  
  }  
}}
```

组合查询

GET product/_search

```
{ "query": {"bool": {"must": [ {  
    "match": {"name": "xiaomi phone"}},{  
    "match_phrase": {"desc": "shouji zhong"}}]}}
```

如果换成filter，效果一样，但性能更好，如果换成must_not则为取反的效果，如果换成should则可以得到类似SQL or的效果，bool关键字可以嵌套。

GET product/_search

```
{"query": {"bool": { "must": [
```



```
{"match": { "name": "xiaomi" }},
```

```
"filter": [ {"range": {"price": { "lte": 1000}}}]}
```

多个短语是一个and关系，本质其实是一个工作流关系

如果是should和must/filter搭配，则should会消极处理自己的条件（即躺平，可以接受任何一个条件都不被满足），此时可以在bool中配置minimum_should_match为N，保证至少N个条件是被满足的。

分析器

doc1: 《Mom's friend》 is an excellent film, but I haven't seen it

doc2: Mr. Ma is an excellent teacher. I'm glad to meet him

搜索词

Teacher ma also thinks 《mother's friends》 is good

term	doc1	doc2
mom's	√	
friend	√	
is	√	√
an	√	√
excellent	√	√
film	√	
but	√	
I => i	√	
haven't=>have	√	
seen=>see	√	
it	√	√
Mr=>mr		√
ma		√
teacher		√
glad		√
to		√
meet		√
him=>he		√

word	docs
Teacher=>teacher	doc2
friends=>friend	doc1
also	
thinks	
mother's=>mother	doc1
is=>is	
good	

GET _analyze

```
{"text":"Mr. Ma is an excellent teacher",
```

```
"analyzer":"english"}
```

换成standard试试?

char_filter

PUT my_index

```
{
  "settings": {
    "analysis": {
      "char_filter": {
        "my_char_filter":{
          "type":"html_strip",
          "escaped_tags":["a"]
        }
      },
      "analyzer": {
        "my_analyzer":{
          "char_filter":["my_char_filter"],
```

```
    "tokenizer":"keyword"
  }
}
}
}
```

PUT my_index

```
{
  "settings": {
    "analysis": {
      "char_filter": {
        "my_char_filter":{
          "type":"mapping",
          "mappings":["垃=>", "圾=>"]
        }
      },
      "analyzer": {
        "my_analyzer":{
          "char_filter":["my_char_filter"],
          "tokenizer":"keyword"
        }
      }
    }
  }
}
```

```
}
```

PUT my_index

```
{  
  "settings": {  
    "analysis": {  
      "char_filter": {  
        "my_char_filter": {  
          "type": "pattern_replace",  
          "pattern": "(\\d{3})\\d{4}(\\d{4})",  
          "replacement": "$1****$2"  
        }  
      },  
      "analyzer": {  
        "my_analyzer": {  
          "char_filter": ["my_char_filter"],  
          "tokenizer": "keyword"  
        }  
      }  
    }  
  }  
}
```

filter

PUT my_index

```
{
  "settings": {
    "analysis": {
      "filter": {
        "my_filter":{
          "type":"synonym_graph",
          "synonyms_path":"analysis/synonym.txt"
        }
      },
      "analyzer": {
        "my_analyzer":{
          "filter":["my_filter"],
          "tokenizer":"keyword"
        }
      }
    }
  }
}
```

PUT my_index

```
{
  "settings": {
    "analysis": {
      "filter": {
        "my_filter":{
```

```

    "type": "synonym",
    "synonyms": ["赵,钱,孙,李=>吴"]
  }
},
"analyzer": {
  "my_analyzer": {
    "filter": ["my_filter"],
    "tokenizer": "standard"
  }
}
}
}
}

```

_analyze中可以使用script脚本，不过这里的script不重要，不会用在生产中，大家了解即可

GET my_index/_analyze

```

{
  "tokenizer": "standard",
  "filter": {
    "type": "condition",
    "filter": "uppercase",
    "script": {
      "source": "token.getTerm().length()<5"
    }
  }
},

```

```
"text":["assas assa sasa dsdsdsdsdsd sdsdsdss"]
}
```

Stopwords用于删除不要的介词和词语,以下为简写

PUT my_index

```
{
  "settings": {
    "analysis": {
      "analyzer": {
        "my_analyzer":{
          "type":"standard",
          "stopwords":["me","you"]
        }
      }
    }
  }
}
```

也可以这样写:

PUT my_index

```
{
  "settings": {
    "analysis": {
      "filter": {
        "my_filter":{
          "type":"stop",
          "stopwords":["me","you"]
        }
      }
    }
  }
}
```

```

    }
  },
  "analyzer": {
    "my_analyzer":{
      "tokenizer":"standard",
      "filter":["my_filter"]
    }
  }
}
}
}

```

自定义分析器

PUT my_index

```

{
  "settings": {
    "analysis": {
      "char_filter": {
        "my_char_filter":{
          "type":"mapping",
          "mappings":["&=>and","|=>or"]
        }
      },
      "filter": {
        "my_filter":{
          "type":"stop",

```



```
    "stopwords":["is","in","a","at"]
  }
},
"tokenizer": {
  "my_tokenizer":{
    "type":"pattern",
    "pattern":[" ,!"]
  }
},
"analyzer": {
  "my_analyzer":{
    "char_filter":["my_char_filter"],
    "filter":["my_filter"],
    "tokenizer":"my_tokenizer",
    "type":"custom"
  }
}
}
```

聚合函数

GET product/_search

```
{
  "size":0,
  "aggs": {
```

```
"max_price": {
  "max": {
    "field": "price"
  }
},
"min_price": {
  "min": {
    "field": "price"
  }
},
"avg_price": {
  "avg": {
    "field": "price"
  }
}
}
```

GET product/_search

```
{
  "size":0,
  "aggs": {
    "price_stats":{
      "stats": {
        "field": "price"
      }
    }
  }
}
```

```
    }  
  }  
}  
}
```

求平均价格最低的type

GET product/_search

```
{  
  "size":0,  
  "aggs": {  
    "type_bucket":{  
      "terms": {  
        "field": "type.keyword"  
      },  
      "aggs": {  
        "price_xxx": {  
          "avg": {  
            "field": "price"  
          }  
        }  
      }  
    },  
    "min_bucket":{  
      "min_bucket": {  
        "buckets_path": "type_bucket>price_xxx"  
      }  
    }  
  }  
}
```

```
}  
}  
}
```

笛卡尔积

GET product/_search

```
{  
  "size":0,  
  "aggs": {  
    "type_bucket":{  
      "terms": {  
        "field": "type.keyword"  
      },  
      "aggs": {  
        "name_xxx": {  
          "terms": {  
            "field": "name.keyword"  
          }  
        }  
      }  
    }  
  }  
}
```

先查询过滤再聚合

GET product/_search

```
{
  "query":{
    "range":{
      "price": {
        "gte": 1000
      }
    }
  },
  "aggs": {
    "type_bucket":{
      "terms": {
        "field": "type.keyword"
      }
    }
  }
}
```